JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.reflect

Class UndeclaredThrowableException

java.lang.Object
  java.lang.Throwable
    java.lang.Exception
      java.lang.RuntimeException
        java.lang.reflect.UndeclaredThrowableException

All Implemented Interfaces:

Serializable

```
public class UndeclaredThrowableException
extends RuntimeException
```

Thrown by a method invocation on a proxy instance if its invocation handler's `invoke` method throws a checked exception (a `Throwable` that is not assignable to `RuntimeException` or `Error`) that is not assignable to any of the exception types declared in the `throws` clause of the method that was invoked on the proxy instance and dispatched to the invocation handler.

An `UndeclaredThrowableException` instance contains the undeclared checked exception that was thrown by the invocation handler, and it can be retrieved with the `getUndeclaredThrowable()` method. `UndeclaredThrowableException` extends `RuntimeException`, so it is an unchecked exception that wraps a checked exception.

As of release 1.4, this exception has been retrofitted to conform to the general purpose exception-chaining mechanism. The "undeclared checked exception that was thrown by the invocation handler" that may be provided at construction time and accessed via the `getUndeclaredThrowable()` method is now known as the *cause*, and may be accessed via the `Throwable.getCause()` method, as well as the aforementioned "legacy method."

Since:

1.3

See Also:

InvocationHandler, Serialized Form

Constructor Summary

Constructors

Constructor	Description
`UndeclaredThrowableException(Throwable undeclaredThrowable)`	Constructs Undeclared with the spe
`UndeclaredThrowableException(Throwable undeclaredThrowable, String s)`	Constructs Undeclared with the spe and a detail

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`Throwable`	`getCause()`	Returns the cause of this exception (the `Throwable` instance wrapped in this `UndeclaredThrowableException`, which may be `null`).
`Throwable`	`getUndeclaredThrowable()`	Returns the `Throwable` instance wrapped in this `UndeclaredThrowableException`, which may be `null`.

Methods inherited from class java.lang.Throwable

addSuppressed, fillInStackTrace, getLocalizedMessage, getMessage, getStackTrace, getSuppressed, initCause, printStackTrace, printStackTrace, printStackTrace, setStackTrace, toString

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

UndeclaredThrowableException

```
public UndeclaredThrowableException(Throwable undeclaredThrowable)
```

Constructs an UndeclaredThrowableException with the specified Throwable.

Parameters:

undeclaredThrowable - the undeclared checked exception that was thrown

UndeclaredThrowableException

```
public UndeclaredThrowableException(Throwable undeclaredThrowable,
                                    String s)
```

Constructs an UndeclaredThrowableException with the specified Throwable and a detail message.

Parameters:

undeclaredThrowable - the undeclared checked exception that was thrown

s - the detail message

Method Detail

getUndeclaredThrowable

```
public Throwable getUndeclaredThrowable()
```

Returns the Throwable instance wrapped in this UndeclaredThrowableException, which may be null.

This method predates the general-purpose exception chaining facility. The Throwable.getCause() method is now the preferred means of obtaining this information.

Returns:

the undeclared checked exception that was thrown

getCause

```
public Throwable getCause()
```

Returns the cause of this exception (the `Throwable` instance wrapped in this `UndeclaredThrowableException`, which may be `null`).

Overrides:

`getCause` in class `Throwable`

Returns:

the cause of this exception.

Since:

1.4

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.